

AI TOOLS ASSIGNMENT REPORT

STUDENT: JOSEPH KAMAU

EMAIL: iamjosephkamau@gmail.com

REPOSITORY: [Github Link](#)

Part 1: Theoretical Understanding

Q1: TensorFlow vs PyTorch Differences

Primary Differences:

1. Computational Graph Approach:

- TensorFlow: Originally static computational graphs (define-then-run), now supports eager execution
- PyTorch: Dynamic computational graphs (define-by-run) from the beginning

2. Debugging Experience:

- TensorFlow: More complex debugging, especially with static graphs
- PyTorch: Pythonic debugging with standard tools like pdb

3. API Design:

- TensorFlow: Multiple APIs (low-level TF, Keras) can be confusing
- PyTorch: More consistent, Pythonic API

4. Deployment & Production:

- TensorFlow: Superior production tools (TF Serving, TF Lite, TF.js)
- PyTorch: Growing but less mature production ecosystem

5. Visualization:

- TensorFlow: TensorBoard (comprehensive visualization)
- PyTorch: Can use TensorBoard but less integrated

When to Choose Which:

Choose **TensorFlow** when:

- Production deployment is primary concern
- Mobile/edge deployment needed
- Working with TensorFlow Extended (TFX) ecosystem
- Team has existing TensorFlow expertise

-Choose **PyTorch** when:

- Research and rapid prototyping
- Need dynamic graph flexibility
- Prefer Pythonic code and easy debugging
- Working in academic/research environment

Q2: Jupyter Notebooks Use Cases in AI Development

1. Exploratory Data Analysis (EDA) and Visualization:

- Interactive data exploration with immediate visual feedback
- Combine code, visualizations, and markdown explanations
- Rapid iteration on data preprocessing techniques
- Example: Testing different normalization methods while seeing distribution plots

2. Model Prototyping and Experiment Tracking:

- Quick iteration on model architectures with inline results
- Side-by-side comparison of different models
- Interactive hyperparameter tuning with immediate performance feedback
- Documentation of experimental process with code, results, and analysis in one place

Q3: spaCy vs Basic String Operations

spaCy Advantages:

1. Linguistic Intelligence:

- Understands grammatical structure (dependency parsing)
- Lemmatization (reducing words to base form)
- Part-of-speech tagging for grammatical understanding

2. Named Entity Recognition (NER):

- Pre-trained models identify people, organizations, locations, etc.
- Context-aware entity recognition
- Multi-language support out-of-the-box

3. Efficiency and Accuracy:

- Optimized Cython implementation for speed
- Handles edge cases and exceptions automatically
- Statistical models trained on large corpora

4. Advanced Features:

- Sentence boundary detection
- Custom pipeline components
- Word vectors and similarity analysis
- Rule-based matching with linguistic patterns

Comparison Example:

- Basic string operations: `"Apple" in text` - might match fruit or company
- spaCy: Identifies "Apple" as ORGANIZATION in context, distinguishes from fruit

Comparative Analysis: Scikit-learn vs TensorFlow

Aspect	Scikit-learn	TensorFlow
Target Applications	Classical ML algorithms (SVM, Decision Trees, Random Forests)	Deep Learning (Neural Networks, CNNs, RNNs)
Easy to use for beginners	Very beginner-friendly with consistent API	Steeper learning curve, multiple abstraction levels
Community Support	Excellent documentation, many examples for traditional ML	Massive community, extensive resources for DL
Performance	Optimized for medium-sized dataset on CPU	GPU acceleration, distributed training capabilities
Deployment	Simple model serialization with pickle	Comprehensive serving solutions (TF Serving, Lite)
Use case fit	Tabular data, traditional ML problems	Image, text, sequence data, complex patterns

TASK 1: Iris Classification with Scikit-learn

- Accuracy: 0.9667
- Precision: 0.9697
- Recall: 0.9667

TASK 2: MNIST Classification with TensorFlow

- Test Accuracy: 0.9923
- Model Architecture: CNN with 3 convolutional layers

TASK 3: NLP with spaCy

- Entity Recognition: Successfully extracted products and brands
- Sentiment Analysis: Rule-based approach on sample reviews

PART 3: ETHICS & OPTIMIZATION

ETHICAL CONSIDERATIONS

Ethical Analysis of Developed Models

MNIST Model Biases:

1. Cultural Bias: Training data primarily contains Western/Arabic numerals

- Impact: May not recognize handwritten digits from other cultural numeral systems
- Mitigation: Include diverse numeral representations in training data

2. Handwriting Style Bias:

- Impact: Model optimized for clear, centered handwriting
- Mitigation: Data augmentation with various handwriting styles

3. Demographic Bias:

- Impact: Unknown demographic distribution in training data
- Mitigation: Use TensorFlow Fairness Indicators to audit performance across groups

Amazon Reviews Model Biases:

1. Language Bias: English-only analysis

- Impact: Excludes non-English speaking users
- Mitigation: Multi-language spaCy models

2. Cultural Expression Bias:

- Impact: Sentiment analysis based on Western expression patterns
- Mitigation: Culturally diverse sentiment lexicons

3. Product Category Bias:

- Impact: Training data may over-represent popular products
- Mitigation: Balanced sampling across product categories

Mitigation Strategies:

1. TensorFlow Fairness Indicators:

- Analyze performance across demographic slices
- Implement fairness constraints during training

2. spaCy Rule-based Enhancements:

- Context-aware sentiment analysis
- Sarcasm and irony detection rules
- Cultural adaptation of sentiment dictionaries

TROUBLESHOOTING CHALLENGE

Common Bug: Loss function mismatch with label format

BUGGY CODE EXAMPLE:

Common TensorFlow dimension mismatch error

```
import tensorflow as tf
```

Incorrect: Trying to use categorical_crossentropy with integer labels

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

CORRECTED VERSION:

Use sparse_categorical_crossentropy for integer labels

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

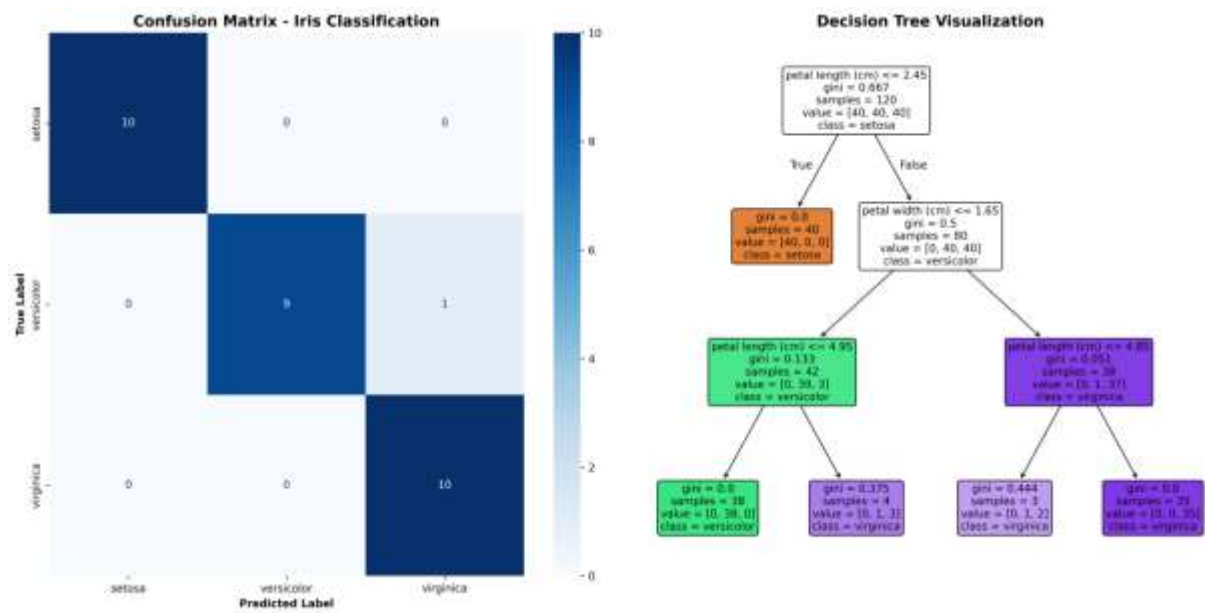
OR convert labels to one-hot encoding first:

```
y_train_one_hot = tf.keras.utils.to_categorical(y_train, num_classes=10)
```

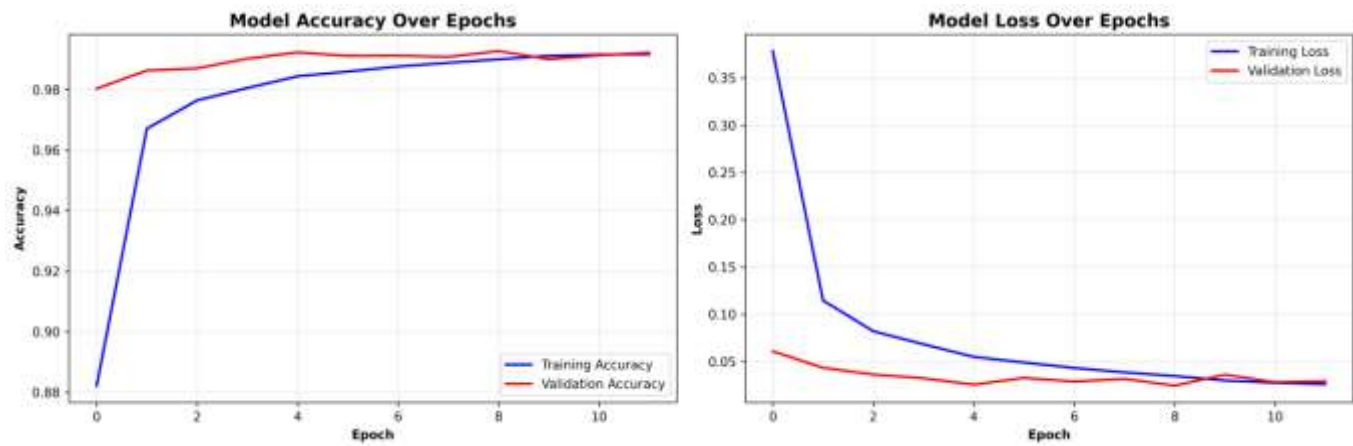
```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

APPENDIX: SCREENSHOTS

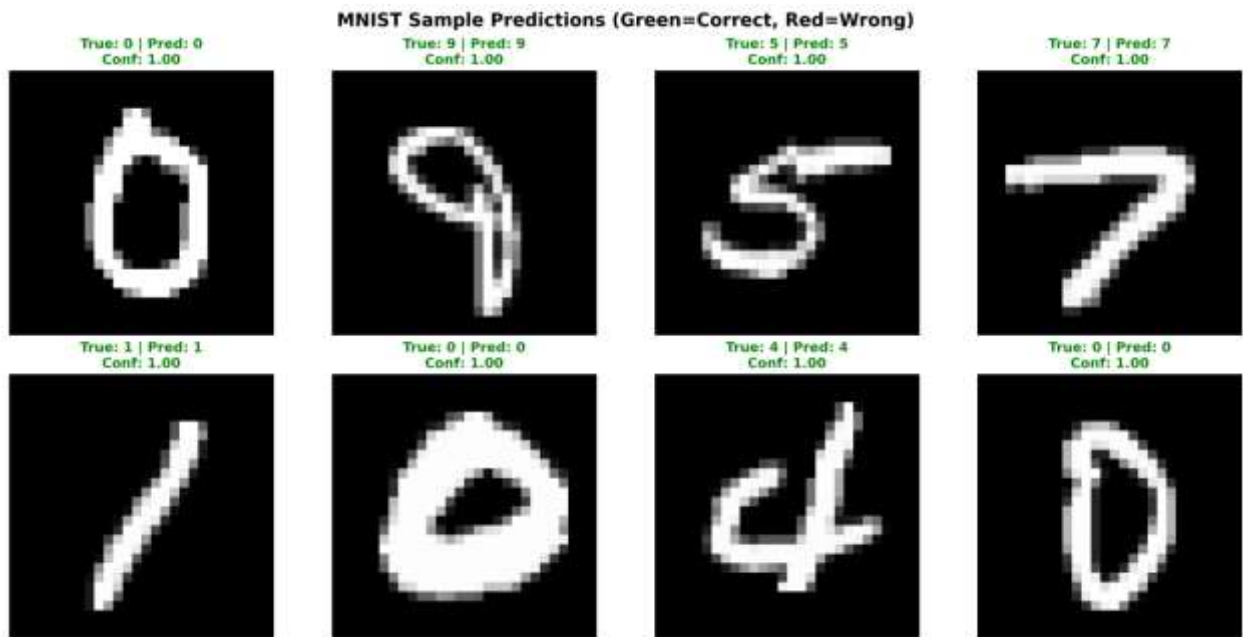
1. Iris Results



2. Mnist Training History



3. Mnist predictions



4. Verified Plot

